
Metal-Sci: A Scientific Compute Benchmark for Evolutionary LLM Kernel Search on Apple Silicon

Anonymous Authors
Anonymous Affiliation
anonymous@example.org

Abstract

We present METAL-SCI, a 10-task benchmark of scientific Apple Silicon Metal compute kernels spanning six optimization regimes (stencils, all-pairs, multi-field Boltzmann, neighbor-list molecular dynamics, multi-kernel PDE, FFT). Each task ships a CPU reference, a roofline-anchored fitness function, and a held-out generalization size. We pair the benchmark with a *lightweight harness for automatic kernel search* that runtime-compile each candidate, scores it against the roofline across multiple sizes, and feeds structured compile and per-size correctness diagnostics back to a frozen LLM driving a (1+1) evolutionary loop. We report matched single-model sweeps of Claude Opus 4.7 and Gemini 3.1 Pro Preview on M1 Pro: in-distribution self-speedups span $1.00\times$ to $10.7\times$. Beyond raw speedup, our central methodological claim is structural: the held-out gate $\Phi_{\mathcal{T}}$ — evaluated once at end-of-run on a configuration the agent never sees during search — functions as a cheap mechanical oversight primitive on this (1+1) coding agent, catching e.g. an Opus template `<uint D> HMC win` that returns wrong samples at unseen $d=24$ where Gemini’s runtime- d variant generalizes at $17.8\times$.

1 Introduction

LLM code-search systems such as FunSearch [12], AlphaEvolve [9], and recent kernel-generation work [11] evaluate language models as *optimisers* of executable artefacts. The choice of artefact matters: KernelBench targets PyTorch ML kernels (GEMM, attention, convolution) on CUDA, where canonical implementations are heavily represented in pretraining data and the optimisation surface is dominated by tiling and warp-level reductions.

Scientific computing exposes a different surface. Stencils are bandwidth-bound and reward halo and temporal blocking. All-pairs interactions are compute-bound and reward register blocking with shared-memory cooperative loads. Lattice Boltzmann ping-pongs nine distribution functions per cell with non-trivial pull-stream indexing. Cell-list molecular dynamics touches irregular memory through atomic counters. Hamiltonian Monte Carlo runs many chains in parallel where register pressure scales with the problem dimension d . Grad-Shafranov solvers chain a max-reduction with a variable-coefficient stencil. 3D FFTs are dominated by data-shuffle/butterfly patterns and twiddle-factor caching rather than tiling. Each pattern stresses a distinct dimension of the compiler/memory hierarchy, and canonical optimizations [2, 10, 14, 1, 3] differ regime-to-regime.

We target the Apple Silicon’s Metal compute pipeline. It is underrepresented in CUDA-centric training data, as frontier models reliably mishandle Metal-specific syntax such as the `[[max_total_threads_per_threadgroup]]` kernel attribute, a simple out-of-distribution generalization test. Its unified memory model removes host-device copy plumbing, enabling sub-second compile-run-verify cycles per candidate that are essential for fast evolutionary loops. And it supports rich simdgroup intrinsics (`simd_max`, `simd_broadcast`) and threadgroup-memory tiling, giving the LLM a non-trivial optimisation surface.

Table 1: The 10 METAL-SCI tasks. “Optimisation lever” names the dominant move an LLM must reach for in that regime; “Train sizes” / “Held-out” are passed to the harness verbatim. $N_x \times N_y$ grids written as N^2 when square; cube edges as N^3 . *saxpy* is a bandwidth smoke-test outside the regime structure, included to validate the harness.

Regime	Task	Optimisation lever	Train sizes	Held-out
R1 stencil	heat2d	halo, temporal blocking	$\{256, 512, 1024\}^2$	768^2
	wave3d	2.5D blocking, register pressure	$\{64, 160, 192\}^3$	128^3
R2 compute	nbody	register tiling, threadgroup cooperative load	$N \in \{256, 1024, 2048\}$	512
	hmc	per-thread state vs. register file	$(d, K) \in \{(8, 16K), (16, 4K), (32, 1K)\}$	$(24, 2K)$
R3 multi-field	lbn	SoA layout, BGK algebraic fold	$\{64, 128, 256\}^2$	192^2
	ising	checkerboard MC, byte-exact verify	$\{256, 1024, 2048\}^2$	1536^2
R4 atomics	lj	cell-list scatter, atomic contention	$N \in \{1.7, 4.1, 10.6\}K$	2744
R5 multi-kernel	gradshaf	in-kernel reduction + var-coef stencil	$\{65, 257, 513\}^2$	129^2
R6 butterfly	fft3d	TG bank conflicts, mixed-radix, <code>simd_shuffle</code>	$\{32, 64, 128\}^3$	256^3
(smoke)	saxpy	DRAM saturation	$\{1, 16, 64\}M$	4M

Contributions. (i) A 10-task scientific compute benchmark over six optimization regimes, each with a CPU reference, roofline-anchored fitness function, and multi-size generalization gate; (ii) a runtime-compiled harness that exposes compile errors, correctness diagnostics, and per-size GPU timing back to the LLM; and (iii) two matched single-model sweeps (Claude Opus 4.7 and Gemini 3.1 Pro) on M1 Pro that operationalise the held-out gate $\Phi_{\mathcal{T}}$ as a cheap mechanical oversight primitive on the (1+1) coding agent: $\Phi_{\mathcal{T}}$ is never folded into any feedback packet seen by the LLM during search and catches confidently-wrong outputs — silent correctness violations and silent regressions — that the in-distribution score $S_{\mathcal{T}}$ alone licenses.

Background and notation. *Apple GPU vocabulary.* A *kernel* is a `.metal`-source GPU function launched from the CPU; a *threadgroup* (TG) is Apple’s name for a CUDA thread block — cooperating threads sharing a scratchpad “threadgroup memory”; a *simdgroup* is the 32-thread SIMD lane group inside a threadgroup (Apple’s name for a CUDA warp); the *System-Level Cache* (SLC) is the CPU/GPU-shared last-level cache (~ 24 MB on M1 Pro) sitting between the on-chip caches and DRAM. *Roofline.* A kernel’s roofline [16] ceiling is the per-chip throughput upper bound implied by its arithmetic intensity (FLOPs per byte transferred). Kernels with low intensity are *bandwidth-bound* and cap at peak DRAM bandwidth (GB/s); high-intensity ones are *compute-bound* and cap at peak FP32 throughput (GFLOPS). We score candidates as a fraction of this hardware ceiling rather than against any hand-written baseline. *Evolution strategy.* ($\mu=1+\lambda=1$), also called (1+1), denotes the simplest evolution strategy: one parent, one mutated child per iteration; the child replaces the parent iff it scores strictly higher.

2 Benchmark tasks

METAL-SCI packages 10 tasks into six *optimisation regimes* (R1–R6, Table 1). The choice of regimes is the benchmark’s central design decision: each regime stresses a structurally distinct dimension of the GPU/memory hierarchy, and its canonical recipe [2, 10, 14, 1, 3] does not transfer to its neighbours. A halo-blocking move that wins R1 is useless on R2 (register tiling), R4 (atomic contention), or R6 (intra-simdgroup butterflies). For an LLM, this is the structural reason recall alone cannot solve the suite: there is no template that wins everywhere, so the model has to actually *recognise the regime* from the seed and reach for the right lever.

Each task ships (a) a Metal seed kernel, (b) a CPU reference with task-specific tolerance, (c) three in-distribution training sizes plus one held-out size, and (d) a per-size roofline ceiling in GFLOPS (compute-bound) or GB/s (bandwidth-bound). Per-task equations, ceilings, and verification details are deferred to App. A; the prose below names the lever each regime tests.

R1 – Regular stencils. Bandwidth-bound updates over a structured grid: a 5-point heat-equation stencil (8 B/cell) and a 7-point leapfrog wave equation (12 B/cell). The lever is halo handling, marching-axis choice, and 2.5D temporal blocking. *wave3d* doubles as a NaN trap — a sign or indexing error compounds over many leapfrog steps, and 10 of Opus’s 13 correctness fails in Sec. 4 land here.

R2 – Compute-bound. $O(N^2)$ pair sums (nbody) and $O(d^2)$ matvec inside an L -step leapfrog (hmc), both running ~ 20 FLOPs per memory transaction so the ceiling is peak FP32 GFLOPS. The lever is register tiling and threadgroup cooperative loads. hmc additionally probes the register-pressure boundary — per-thread $\mathbf{q}, \mathbf{p}, \mathbf{f}$ at $d=32$ is ~ 512 B — and verifies correctness statistically (sample mean and Frobenius covariance error vs. the target Gaussian).

R3 – Multi-field, exotic memory. 1bm is a D2Q9 Lattice Boltzmann (pull-stream + BGK collision) with nine distribution fields per cell at 72 B/cell traffic; the lever is SoA layout, push/pull streaming choice, and algebraic factorisations of the BGK relaxation (App. C dissects an FMA fold Opus discovered). ising is 2D Ising checkerboard Metropolis MC at 2 B/site; a precomputed accept-probability table and a counter-based Murmur-fmix32 PRNG yield bit-exact CPU/GPU agreement, so verification reduces to byte-equality on the spin array.

R4 – Irregular memory and atomics. Lennard-Jones MD with a cell-list spatial hash. Three kernels per step (clear_cells / build_cells / step); build_cells is an atomic scatter onto per-cell occupancy counters and the force kernel walks 27 neighbour cells with minimum-image periodic wrap. The lever is load balancing under uneven cell occupancy and atomic-contention mitigation.

R5 – Multi-kernel reductions. Picard iteration for the Grad-Shafranov fixed-boundary plasma equilibrium. Each outer step dispatches a max-reduction ($\psi_{\text{axis}} = \max \psi$) followed by a variable-coefficient 5-point stencil with a nonlinear source. The lever is the choice of in-kernel reduction strategy — single-threadgroup vs. simdgroup-tree — and how cleanly the two kernels compose across the dispatch boundary.

R6 – Data-shuffle / butterfly. 3D complex-to-complex forward FFT, dispatched as three per-axis 1D FFTs with two ping-ponged buffers. Unlike R1/R2, the optimisation surface is *data movement* rather than arithmetic: bit-reversal vs. Stockham auto-sort, twiddle-factor caching, mixed-radix (radix-4, radix-8) butterflies for fewer barriers, and intra-simdgroup permutes via simd_shuffle_xor. The per-axis stride asymmetry (stride 1 along x vs. N and N^2 along y, z) makes threadgroup-memory bank-conflict avoidance a separate sub-lever.

3 Harness Design

Compile and dispatch. The harness uses PyObjC’s Metal bindings and runtime-compiles .metal source via MTLDevice.newLibraryWithSource, avoiding the offline xcrun metal toolchain; compile errors are returned as structured strings to the LLM. Buffers are allocated in unified memory; numpy views alias buffer contents with a single copy on read-back. All dispatches for a multi-size run share one MTLCommandBuffer, and timings come from GPUEndTime – GPUStartTime (3 warmup, 10 timed, median reported). The chip is detected from sysctl and looked up in a per-family table (M1 through M4) for peak FP32 GFLOPS and DRAM bandwidth; all runs here are on Apple M1 Pro (4500 GFLOPS, 200 GB/s).

Notation. A task $\mathcal{T}=(\sigma_{\mathcal{T}}, \Sigma_{\mathcal{T}}, \sigma_{\mathcal{T}}^*, c_{\mathcal{T}})$ packages a naive correct seed kernel $\sigma_{\mathcal{T}}$ (Metal source), three training sizes $\Sigma_{\mathcal{T}}=\{s_1, s_2, s_3\}$, one held-out size $\sigma_{\mathcal{T}}^* \notin \Sigma_{\mathcal{T}}$, and a per-size roofline $c_{\mathcal{T}}(s)$ in GFLOPS or GB/s. Evaluating a candidate κ at size s produces a correctness flag $\chi_{\mathcal{T}}(\kappa, s) \in \{0, 1\}$ (CPU reference within tolerance or not) and an achieved throughput $a_{\mathcal{T}}(\kappa, s)$; write $f_{\mathcal{T}}(\kappa, s) = a_{\mathcal{T}}(\kappa, s) / c_{\mathcal{T}}(s)$ for the per-size fraction-of-ceiling.

Scoring. The in-distribution score is the geometric mean of $f_{\mathcal{T}}$ over training sizes, gated on correctness on *every* size:

$$S_{\mathcal{T}}(\kappa) = \left(\prod_{s \in \Sigma_{\mathcal{T}}} f_{\mathcal{T}}(\kappa, s) \right)^{1/|\Sigma_{\mathcal{T}}|} \cdot \prod_{s \in \Sigma_{\mathcal{T}}} \chi_{\mathcal{T}}(\kappa, s). \quad (1)$$

The hard gate ($S_{\mathcal{T}}=0$ on any tolerance failure) prevents trading correctness for speed; the gmean across sizes discourages overfit to one regime. The held-out gate, run only on the run’s incumbent at end-of-run, is the analogous quantity at the unseen size: $\Phi_{\mathcal{T}}(\kappa) = f_{\mathcal{T}}(\kappa, \sigma_{\mathcal{T}}^*) \cdot \chi_{\mathcal{T}}(\kappa, \sigma_{\mathcal{T}}^*)$. $S_{\mathcal{T}}$ is the agent’s optimisation target; $\Phi_{\mathcal{T}}$ is the external oversight signal it never sees.

Table 2: Matched 10/15/25-iter sweeps on Apple M1 Pro: O = claude-opus-4-7, G = gemini-3.1-pro-preview. *In-dist.* $\times$ = best / seed, gmean over three training sizes; correctness is a hard gate. *Held-out frac.* = achieved/ceiling at the unseen size; *held-out* $\times$ = best / seed at that size. **Bold** marks regressions ($< 0.95\times$) and the HMC correctness fail.

Task	In-dist. $\times$		Held-out frac.		Held-out $\times$		Outcome
	O	G	O	G	O	G	
saxpy	1.25	1.00	78%	105%	0.79	1.06	saturated
heat2d	1.00	1.03	79%	84%	0.80	0.85	saturated
wave3d	1.26	1.00	99%	98%	1.02	1.01	saturated
ising	1.13	1.00	11%	12%	0.95	0.99	flat
fft3d	1.03	1.19	40%	45%	1.02	1.15	generalises
nbody	2.83	2.00	1.7%	1.6%	2.21	2.17	generalises
gradshaf	1.89	2.89	5.5%	8.6%	2.13	3.31	generalises
lj	1.77	1.98	0.25%	0.48%	1.16	2.19	generalises
lbm	1.46	1.06	44%	54%	0.41	0.50	both regress (TG geom.)
hmc	10.6	10.7	—	9.85%	FAIL	17.8	O wrong at $d=24$; G clean

Evolution loop. A frozen LLM $\mathcal{M}$ acts as a kernel synthesiser. Given a task-spec system prompt $p_{\mathcal{T}}$ and a feedback packet $\mathcal{F}_k$ summarising the previous candidate, the incumbent, and a short per-iteration history, $\mathcal{M}$ emits a Metal source

$$\kappa_{k+1} = \mathcal{M}(p_{\mathcal{T}}, q(\kappa_k, \kappa_k^*, \mathcal{F}_k)), \quad \kappa_0 = \sigma_{\mathcal{T}}, \quad (2)$$

which the harness compiles, dispatches across $\Sigma_{\mathcal{T}}$, and scores; the strict (1+1) rule promotes $\kappa_{k+1}^* = \kappa_{k+1}$ iff $S_{\mathcal{T}}(\kappa_{k+1}) > S_{\mathcal{T}}(\kappa_k^*)$, otherwise $\kappa_{k+1}^* = \kappa_k^*$. We formalise this compile-evaluate-promote (1+1) loop in Alg. 1 (App. B). Compile, pipeline, and per-size correctness errors are returned inside $\mathcal{F}_{k+1}$ as structured strings (the violating size, the error metric and value, and the compiler diagnostic if any). After K iterations the run terminates; both $S_{\mathcal{T}}(\kappa_K^*)$ and the held-out $\Phi_{\mathcal{T}}(\kappa_K^*)$ are reported. A single `call_llm` bridge dispatches Claude via the Agent SDK and Gemini via `google-genai`.

4 Experiments

We run two matched single-model sweeps on Apple M1 Pro — claude-opus-4-7 (*O*) and gemini-3.1-pro-preview (*G*) — over the ten tasks at the same per-task iteration budget (10 each except lbm at 25 and wave3d at 15), $\mu=1+\lambda=1$, no human prompt intervention. Table 2 reports each model’s in-distribution self-speedup (best / seed, gmean over three training sizes) and a held-out evaluation in which the unmodified seed and each incumbent best are run on a single unseen problem size declared by the task spec (Sec. 2).

In-distribution. Self-speedups span $1.00\times$ to $10.7\times$. The HMC step (Fig. 1) is the high end for *both models*: each independently introduces a `template <uint D>` worker dispatched on runtime d that takes $d=8$ from ~ 120 to ~ 970 GFLOPS by enabling full unroll of the inner Aq matvec, and the two top scores agree to within 1.4% (O 0.0932 vs G 0.0870). Outside HMC the models split: Opus wins clearly on saxpy (1.25 vs 1.00), nbody (2.83 vs 2.00), lbm (1.46 vs 1.06), and wave3d (1.26 vs 1.00); Gemini wins on gradshaf (2.89 vs 1.89), fft3d (1.19 vs 1.03), and lj (1.98 vs 1.77). The pattern correlates with the type of optimisation lever: “tune the same algorithm tighter” (BW saturation, BGK fold, leapfrog ILP) favours Opus, while “find a different algorithm” (simdgroup-tree reduction, twiddle caching, neighbour-list reorganisation) favours Gemini (see App. C for code-level diffs on lbm and fft3d). Saturated tasks (saxpy, heat2d, wave3d) sit above 78% of effective DRAM ceiling on the seed; saxpy and heat2d primarily validate the harness, leaving the search loop little to do — and heat2d shows the score is a *hard* signal: on Opus zero candidates strictly dominated the seed across all three sizes and the incumbent stayed at iter 0. Gemini had *zero* correctness failures across its 95 fresh-run candidates; Opus had 13 across the same candidate budget (notably wave3d 10/15: multi-step leapfrog amplifies any sign or indexing error into NaN).

Held-out generalisation is sharper than in-distribution — and the two models overfit asymmetrically. Three populations separate cleanly. (i) *Generalises*: nbody, fft3d, gradshaf, lj. Gradshaf

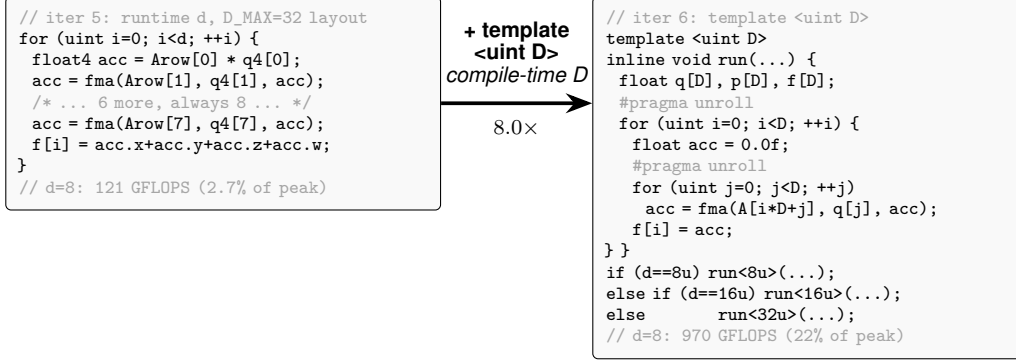


Figure 1: Paradigmatic candidate evolution: HMC, Opus 4.7, iter 5 $\rightarrow$ 6. Both Opus and Gemini independently arrive at this structural change. The lever is one declaration — `template <uint D>` with runtime-dispatched instantiations on d . Iter 5 had a manually-unrolled float4 inner loop against a fixed $D_{\max}=32$ layout (over-computing at $d=8$, plus a 4-way horizontal sum); iter 6 sizes q, p, f exactly to D so both loops fully unroll into scalar FMAs — moving $d=8$ from 121 to 970 GFLOPS in one iteration after five had stalled at 2–4% of peak. Opus enumerated $D \in \{8, 16, 32\}$ and broke correctness at the held-out $d=24$; Gemini kept a runtime- d fallback and generalised cleanly.

is the only task where both models match or beat their in-distribution speedup ($2.13\times$ Opus, $3.31\times$ Gemini); on l_j only Gemini extrapolates cleanly ($2.19\times$ at $N=2744$, above its in-distribution $1.98\times$), while Opus drops from $1.77\times$ to $1.16\times$. `fft3d 2563` extrapolates past the largest training size 128^3 for both models. (ii) *Saturated*: `saxpy`, `heat2d`, `wave3d`, `ising` — nothing to win. (iii) *Overfits*, with the most consequential disagreement between the two models. **HMC** is the sharpest case: Opus’s template specialisation dispatches `if (d==8) run<8>() ... else run<32>()`, so $d=24$ lands in the $D=32$ branch, per-thread `q[32]`, `p[32]` and the unrolled matvec process 32 entries against 24-entry data, and sample covariance lands $\sim 10\sigma$ off target. Gemini arrived at the same template- D speedup in-distribution but kept a runtime- d leapfrog around it, which generalises to $d=24$ at 9.85% of FP32 peak — $17.8\times$ the seed. **LBM** regresses for both ($0.41\times$ Opus, $0.50\times$ Gemini) at the held-out 192^2 grid: each pins a fixed `[[max_total_threads_per_threadgroup]]` geometry tuned for the $\{64, 128, 256\}^2$ training sizes; on a non-multiple of 32 the geometry wastes threads at the boundary, and the BGK algebraic fold cannot recover the loss. **Gemini is never strictly worse than Opus on held-out across the ten tasks**: it is the only model correct at HMC $d=24$, posts a higher held-out speedup on six of the remaining nine tasks, and ties on the other three. The LBM 256^2 result above 100% of nominal ceiling reflects working-set residency in the system-level cache: the 8 B/cell roofline is DRAM-only.

Recurring Metal-grammar failures. Compile-error patterns are similar across models. `[[max_total_threads_per_threadgroup(N)]]` is mis-placed (after the parameter list, or as a standalone statement, instead of on the `kernel void` declaration) on Opus across five tasks — reproduced from earlier opus-4-6 runs. `half` is reserved as MSL’s fp16 type, breaking `uint half = N > 1u;`; C++ lambdas are unsupported. The two sweeps differ in volume rather than kind: 12/110 compile fails for Opus vs 22/95 for Gemini, with Gemini exploring more aggressively per iteration but never breaking correctness, and Opus producing more correct-but-slow candidates. Gemini’s wall-time per iteration is 5–10 $\times$ Opus’s at matched budget, the cost of that wider exploration.

5 Discussion

The held-out gate as agent oversight. The benchmark’s $(1+1)$ loop is, modulo terminology, an autonomous coding agent: it reads a Metal source, edits it, runs it, and self-promotes its own outputs based on an internal fitness signal. A human merging the agent’s incumbent into a downstream code-base sees only the in-distribution score $S_{\mathcal{T}}$ the agent reports — and $S_{\mathcal{T}}$ is gameable in two ways the held-out gate $\Phi_{\mathcal{T}}$ (Sec. 3) catches. **Silent correctness violation.** On HMC, Opus’s incumbent hits $10.6\times$ with all in-distribution checks green; held out at $d=24$ — a problem dimension that sits between two training values and is not flagged as out-of-scope — the same code returns samples

whose covariance is off by $\sim 10\sigma$. A user who trusted the reported number would ship a sampler that looks calibrated and isn't. **Silent regression.** On LBM both models report in-distribution wins ($1.46\times$, $1.06\times$) that flip to slowdowns at the held-out grid ($0.41\times$, $0.50\times$); the agent confidently labels its output as "improvement" on a configuration only $1.5\times$ from the training sizes. Both failures share a structural shape: the agent specialised against the visible workload and reported a number that does not carry over. A single auxiliary problem instance per task — one extra dispatch, seconds of GPU time — is enough to surface both. This reframes the contribution toward the workshop's verifiability/oversight agenda: METAL-SCI's value lies less in providing a hard benchmark and more in instantiating a cheap, mechanical trust contract on top of an autonomous coding agent. The roofline anchor, per-size tolerance, and geometric mean inside S_T are the agent's scoring machinery; Φ_T — evaluated once on σ_T^* at end-of-run, never folded into any feedback packet $\mathcal{F}_k$ — is the lightweight oversight primitive that lets a human (or a downstream automated reviewer) catch confidently-wrong agent code before deployment.

Other observations. A roofline-anchored score answers "how close are we to the hardware?" in physical units; the gmean across sizes is hard to game by overfitting one regime. The Opus–Gemini asymmetry is concrete: at matched iteration budgets the in-distribution scores are close, but Gemini's optimisations preserve correctness across the held-out size on every task while Opus's break it on one. If kernels will be deployed across configurations the training set did not cover, that robustness is worth the $5\text{--}10\times$ wall-time premium. Apple Silicon's unified memory halves the host-side scaffolding compared to CUDA, enabling sub-second compile-run-verify cycles per candidate — which matters more than it sounds: an evolutionary loop with tens of candidates needs a fast *har-ness*, not a fast kernel. Metal's smaller, quirkier surface also surfaces OOD failures of CUDA-trained LLMs on tasks that are otherwise textbook.

Limitations and future work. The static per-chip ceilings do not account for SLC residency at small sizes (see the LBM 256² result above); a workload-aware roofline [16] per task and per size would tighten the score. The single-population (1+1) loop plateaus within 10–25 iterations on most tasks — an island-model or FunSearch-style [12] archive with a novelty signal [9] is the natural next step, particularly for the irregular-memory tasks. Future tasks include sparse linear algebra (SpMV, CG) and cross-chip generalisation (evolve on M2 Pro, evaluate on M3 Max).

References

- [1] J. A. Anderson, C. D. Lorenz, and A. Travesset. General purpose molecular dynamics simulations fully implemented on graphics processing units. *Journal of computational physics*, 227(10):5342–5359, 2008.
- [2] K. Datta, M. Murphy, V. Volkov, S. Williams, J. Carter, L. Oliker, D. Patterson, J. Shalf, and K. Yelick. Stencil computation optimization and auto-tuning on state-of-the-art multicore architectures. In *Proceedings of the 2008 ACM/IEEE Conference on Supercomputing*, SC '08. IEEE Press, 2008.
- [3] N. K. Govindaraju, B. Lloyd, Y. Dotsenko, B. Smith, and J. Manferdelli. High performance discrete fourier transforms on graphics processors. *SC*, 8:1–12, 2008.
- [4] P. Guo, C. Zhu, S. Chen, F. Liu, X. Lin, Z. Lu, and Q. Zhang. Evoengineer: Mastering automated cuda kernel code evolution with large language models. *arXiv preprint arXiv:2510.03760*, 2025.
- [5] S. Kalade and G. Schelle. NPUEval: Optimizing npu kernels with llms and open source compilers. *arXiv preprint arXiv:2507.14403*, 2025.
- [6] R. T. Lange, Q. Sun, A. Prasad, M. Faldor, Y. Tang, and D. Ha. Towards robust agentic cuda kernel benchmarking, verification, and optimization. *arXiv preprint arXiv:2509.14279*, 2025.
- [7] J. Li, S. Li, Z. Gao, Q. Shi, Y. Li, Z. Wang, J. Huang, W. WangHaojie, J. Wang, X. Han, et al. Tritonbench: Benchmarking large language model capabilities for generating triton operators. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 23053–23066, 2025.
- [8] J. Nie, H. Wu, Y. Lai, Z. Cao, C. Zhang, B. Lou, E. Wang, J. Cheng, T. M. Jones, R. Mullins, et al. Kernelcraft: Benchmarking for agentic close-to-metal kernel generation on emerging hardware. *arXiv preprint arXiv:2603.08721*, 2026.

- [9] A. Novikov, N. Vĩ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. Ruiz, A. Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [10] L. Nyland, M. Harris, and J. Prins. Fast n-body simulation with cuda. *gpu gems* 3, 2007.
- [11] A. Ouyang, S. Guo, S. Arora, A. L. Zhang, W. Hu, C. Re, and A. Mirhoseini. Kernelbench: Can LLMs write efficient GPU kernels? In *Forty-second International Conference on Machine Learning*, 2025.
- [12] B. Romera-Paredes, M. Barekatin, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 2023.
- [13] M. Saroufim, J. Wang, B. Maher, S. Paliskara, L. Wang, S. Sefati, and M. Candales. Backendbench: An evaluation suite for testing how well llms and humans can write pytorch backends, 2025.
- [14] M. Schönherr, K. Kucher, M. Geier, M. Stiebler, S. Freudiger, and M. Krafczyk. Multi-thread implementations of the lattice boltzmann method on non-uniform grids for cpus and gpus. *Computers & Mathematics with Applications*, 61(12):3730–3743, 2011.
- [15] Z. Wen, Y. Zhang, Z. Li, Z. Liu, L. Xie, and T. Zhang. Multikernelbench: A multi-platform benchmark for kernel generation. *arXiv e-prints*, pp. *arXiv–2507*, 2025.
- [16] S. Williams, A. Waterman, and D. Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

A Task formulations

This appendix gives the per-task equations, ceilings, and tolerances summarised in Section 2. Tasks are grouped by the regimes R1–R6 of Table 1; training and held-out sizes match the table.

A.1 R1: Regular stencils

heat2d. Two-dimensional heat equation, 5-point stencil on $(N_x \times N_y)$ grid with Dirichlet boundaries. Per timestep, per interior cell:

$$u_{i,j}^{n+1} = u_{i,j}^n + \alpha(u_{i-1,j}^n + u_{i+1,j}^n + u_{i,j-1}^n + u_{i,j+1}^n - 4u_{i,j}^n). \quad (3)$$

Bandwidth-bound at 8 B/cell.

wave3d. Three-dimensional acoustic wave equation, 7-point isotropic Laplacian, leapfrog in time:

$$u_{i,j,k}^{n+1} = 2u_{i,j,k}^n - u_{i,j,k}^{n-1} + \alpha(u_{i\pm 1,j,k}^n + u_{i,j\pm 1,k}^n + u_{i,j,k\pm 1}^n - 6u_{i,j,k}^n), \quad (4)$$

where each $\pm$ expands to two terms. CFL stability requires $\alpha = (c\Delta t/\Delta x)^2 < 1/3$ (we use 0.18). 12 B/cell unique DRAM traffic.

A.2 R2: Compute-bound

nbody. All-pairs gravitational N -body with leapfrog integration. Per body i per step:

$$\mathbf{a}_i = G \sum_j m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{(\|\mathbf{r}_j - \mathbf{r}_i\|^2 + \varepsilon^2)^{3/2}}, \quad \mathbf{v} \leftarrow \mathbf{v} + \mathbf{a}\Delta t, \quad \mathbf{r} \leftarrow \mathbf{r} + \mathbf{v}\Delta t. \quad (5)$$

Self-interaction is masked by softening ε . ~ 20 FLOPs per pair; ceiling at peak FP32 GFLOPS.

hmc. Hamiltonian Monte Carlo on an anisotropic Gaussian target, $U(q) = \frac{1}{2}q^\top Aq$ with $A = \Sigma^{-1}$. One thread per chain; many chains in parallel. Each step draws $p \sim \mathcal{N}(0, \hat{I})$ and runs L leapfrog inner steps with stepsize ε ,

$$p \leftarrow p - \frac{\varepsilon}{2}Aq, \quad q \leftarrow q + \varepsilon p, \quad p \leftarrow p - \varepsilon Aq, \quad \dots, \quad p \leftarrow p - \frac{\varepsilon}{2}Aq, \quad (6)$$

followed by a Metropolis accept/reject with $\log u_{\text{acc}} < -\Delta H$. Correctness is verified statistically (sample mean and Frobenius covariance error vs. the target). The three training sizes $(d, K) \in \{(8, 16K), (16, 4K), (32, 1K)\}$ probe the register-pressure boundary; at $d=32$ per-thread state (~ 512 B) competes with the register file.

A.3 R3: Multi-field, exotic memory

lbm. D2Q9 Lattice Boltzmann, fused pull-stream + BGK collision, periodic BC. With velocity table $\mathbf{c}_k$ and weights w_k , per cell per step:

$$f_k^{\text{str}}(\mathbf{x}) = f_k^{\text{in}}(\mathbf{x} - \mathbf{c}_k), \quad \rho = \sum_k f_k^{\text{str}}, \quad \mathbf{u} = \frac{1}{\rho} \sum_k \mathbf{c}_k f_k^{\text{str}}, \quad (7)$$

$$f_k^{\text{eq}} = w_k \rho \left(1 + 3(\mathbf{c}_k \cdot \mathbf{u}) + \frac{9}{2}(\mathbf{c}_k \cdot \mathbf{u})^2 - \frac{3}{2}\|\mathbf{u}\|^2 \right), \quad (8)$$

$$f_k^{\text{out}} = f_k^{\text{str}} - \tau^{-1}(f_k^{\text{str}} - f_k^{\text{eq}}). \quad (9)$$

Storage is SoA: $f[k N_x N_y + j N_x + i]$. 72 B/cell DRAM traffic.

ising. Two-dimensional Ising model, checkerboard Metropolis Monte Carlo on a periodic $N_x \times N_y$ lattice with $\sigma \in \{\pm 1\}$ stored as int8. Per attempt at site (i, j) :

$$h = \sigma_{i\pm 1, j} + \sigma_{i, j\pm 1} \in \{-4, \dots, 4\}, \quad \Delta E = 2J\sigma h, \quad \text{accept} \iff u < \exp(-\beta \Delta E). \quad (10)$$

A precomputed five-entry p_{accept} table and a counter-based Murmur-fmix32 PRNG yield bit-exact CPU/GPU agreement; verification is byte-equality on the spin array. 2 B/site/sweep ceiling.

A.4 R4: Irregular memory and atomics

lj. Lennard-Jones molecular dynamics with a cell-list spatial hash. Three kernels per step — `clear_cells`, `build_cells` (atomic scatter), `step` (27-neighbour-cell pair force + integration):

$$\mathbf{F}_i = \sum_{j \in \mathcal{N}(i)} -24(2r_{ij}^{-12} - r_{ij}^{-6}) r_{ij}^{-2} (\mathbf{r}_j - \mathbf{r}_i), \quad r_{ij} < r_{\text{cut}} = 2.5, \quad (11)$$

with minimum-image periodic wrap. Cell occupancy is built via `atomic_fetch_add` on a per-cell counter.

A.5 R5: Multi-kernel reductions

gradshaf. Grad-Shafranov fixed-boundary equilibrium via Picard iteration, two kernels per outer step: an interior max-reduction $\psi_{\text{axis}} = \max_{(i,j) \in \text{int}} \psi$, then a variable-coefficient 5-point stencil with nonlinear source:

$$\psi_{\text{norm}} = \psi / \psi_{\text{axis}}, \quad J = R p_{\text{axis}} \cdot 4\psi_{\text{norm}}(1 - \psi_{\text{norm}}) \cdot \mathbf{1}[0 < \psi_{\text{norm}} < 1], \quad (12)$$

$$\Delta^* \psi = a_W \psi_W + a_E \psi_E + a_N \psi_N + a_S \psi_S + a_C \psi_C, \quad (13)$$

$$\psi^{n+1} = \psi^n + \omega(-\mu_0 R J - \Delta^* \psi) / a_C, \quad (14)$$

with R -dependent $a_{W,E} = 1/dR^2 \pm 1/(2RdR)$, $a_{N,S} = 1/dZ^2$, $a_C = -2(1/dR^2 + 1/dZ^2)$.

A.6 R6: Data-shuffle / butterfly

fft3d. 3D complex-to-complex forward FFT, fp32, on a power-of-two cube of side N . Convention is unnormalised (matches `numpy.fft.fftn`); storage is row-major `float2[N][N][N]`. Three named kernels — `fft3d_x`, `fft3d_y`, `fft3d_z` — are dispatched in sequence with two ping-ponged buffers, each performing a length- N 1D FFT per threadgroup of N cooperating threads:

$$Y[k_1, k_2, k_3] = \sum_{n_1, n_2, n_3=0}^{N-1} X[n_1, n_2, n_3] \exp\left(-\frac{2\pi i}{N}(k_1 n_1 + k_2 n_2 + k_3 n_3)\right). \quad (15)$$

Sizes $N \in \{32, 64, 128\}$ cover three working-set regimes: 32^3 (~ 256 KB) is SLC-resident and compute-bound; 128^3 (~ 16 MB) DRAM-binds. $\sim 5N \log_2 N$ FLOPs per 1D FFT; effective DRAM traffic 96 B/cell across the three axis passes (16 B read + 16 B write per pass). Verification is max-norm against `numpy.fft.fftn` on a fixed seeded Gaussian input, tolerance $10^{-3} + 10^{-3}\|Y\|_\infty$.

Algorithm 1 METAL-SCI evolution loop ($\mu=1+\lambda=1$).

Require: task $\mathcal{T}=(\sigma_{\mathcal{T}}, \Sigma_{\mathcal{T}}, \sigma_{\mathcal{T}}^*, c_{\mathcal{T}})$, LLM $\mathcal{M}$, system prompt $p_{\mathcal{T}}$, iterations K
Ensure: incumbent κ_K^* , in-dist score $S_{\mathcal{T}}(\kappa_K^*)$, held-out $\Phi_{\mathcal{T}}(\kappa_K^*)$

```
1:  $\kappa_0^* \leftarrow \sigma_{\mathcal{T}}; \mathcal{F}_0 \leftarrow \text{EVALUATE}(\sigma_{\mathcal{T}}, \mathcal{T})$  ▷ seed is initial incumbent
2: for  $k = 1, \dots, K$  do
3:    $\kappa_k \leftarrow \mathcal{M}(p_{\mathcal{T}}, q(\kappa_{k-1}, \kappa_{k-1}^*, \mathcal{F}_{k-1}))$  ▷ propose
4:    $r_k \leftarrow \text{EVALUATE}(\kappa_k, \mathcal{T})$ 
5:    $\mathcal{F}_k \leftarrow (\kappa_k, r_k)$  ▷ structured feedback for next iter
6:   if  $r_k.\text{score}$  defined  $\wedge r_k.\text{score} > S_{\mathcal{T}}(\kappa_{k-1}^*)$  then
7:      $\kappa_k^* \leftarrow \kappa_k$  ▷ (1+1) promote
8:   else
9:      $\kappa_k^* \leftarrow \kappa_{k-1}^*$ 
10:  end if
11: end for
12:  $\Phi_{\mathcal{T}}(\kappa_K^*) \leftarrow f_{\mathcal{T}}(\kappa_K^*, \sigma_{\mathcal{T}}^*) \cdot \chi_{\mathcal{T}}(\kappa_K^*, \sigma_{\mathcal{T}}^*)$  ▷ never given to  $\mathcal{M}$ 
13: return  $\kappa_K^*, S_{\mathcal{T}}(\kappa_K^*), \Phi_{\mathcal{T}}(\kappa_K^*)$ 

14: function  $\text{EVALUATE}(\kappa, \mathcal{T})$ 
15:    $\text{lib}, e_c \leftarrow \text{COMPILE}(\kappa)$ 
16:   if  $e_c \neq \emptyset$  then return  $(\text{compile\_fail}, e_c)$ 
17:   end if
18:   for  $s \in \Sigma_{\mathcal{T}}$  do
19:      $(\chi_s, a_s) \leftarrow \text{RUN}(\text{lib}, s)$ 
20:     if  $\chi_s = 0$  then return  $(\text{correct\_fail}, s, \text{error metric})$ 
21:   end if
22: end for
23:    $S \leftarrow (\prod_{s \in \Sigma_{\mathcal{T}}} a_s / c_{\mathcal{T}}(s))^{1/|\Sigma_{\mathcal{T}}|}$ 
24:   return  $(\text{score}: S, \{a_s / c_{\mathcal{T}}(s)\}_{s \in \Sigma_{\mathcal{T}}})$ 
25: end function
```

B Evolution loop pseudocode

Algorithm 1 formalises the harness of Sec. 3. The EVALUATE subroutine is the compile–run–score pipeline: it returns either a structured failure (compile or per-size correctness, with the violating size s and the error metric so $\mathcal{M}$ can correct course inside $\mathcal{F}_{k+1}$) or a successful score $S_{\mathcal{T}}(\kappa)$ together with per-size fractions-of-ceiling. The held-out $\Phi_{\mathcal{T}}(\kappa_K^*)$ is computed once at end-of-run on the unseen size $\sigma_{\mathcal{T}}^*$ — never returned in any $\mathcal{F}_k$ — and is the oversight signal of Sec. 4.

C Code-level evidence for the in-distribution split

The comparative claim in Section 4 is mechanistic — Opus wins by tightening the same algorithm, Gemini wins by reaching for a different one. Figure 1 instantiates that claim on HMC. This appendix instantiates it on two more tasks — one Opus-favoured (lbn), one Gemini-favoured (fft3d) — with the actual diff between each model’s incumbent best.

C.1 LBM: Opus tightens BGK with FMA folds and a pinned threadgroup

Both models share the seed’s pull-stream + BGK structure. The diff is local to the collision step and to the kernel attribute (Fig. 2). Opus precomputes $A = \text{fma}(-1.5, \|\mathbf{u}\|^2, 1)$ once per cell, factors the per-direction equilibrium as $A + c \cdot u(3 + 4.5c \cdot u)$ — two FMAs — and folds the relaxation into a third $\text{fma}(1-\omega, f_k, \omega W_k \rho t)$, yielding nine manually unrolled blocks. It also pins `[[max_total_threads_per_threadgroup(64)]]` on the kernel declaration, picking a 32×2 tile aligned to the simdgroup width. Gemini keeps the textbook BGK formula $w_k \rho(1+3c \cdot u+4.5(c \cdot u)^2-1.5\|\mathbf{u}\|^2)$ inside a `#pragma unroll` for (`k=0...8`), no FMA folds, no A -extraction, no threadgroup pin. The two are competitive at 64^2 (O 0.34, G 0.32) and Gemini actually wins at 128^2 (O 0.47, G 0.51); Opus pulls ahead at 256^2 (O 1.22, G 1.03), the cache-resident

```

// Opus: BGK fold + pinned geometry
[[max_total_threads_per_threadgroup(64)]]
kernel void lbm_step(...) {
    // pull-stream f0..f8, moments rho, ux, uy ...
    float A = fma(-1.5f, usq, 1.0f);
    float orWS = (omega * W_S) * rho;
    // k=5: cu = ux+uy
    {
        float cu = ux + uy;
        float t = A + cu * fma(4.5f, cu, 3.0f);
        q5[idx] = fma(one_m_w, f5, orWS * t);
    }
    // 8 more directions, same shape ...
}

```

```

// Gemini: textbook BGK in unrolled loop
kernel void lbm_step(...) {
    // pull-stream f[k], moments rho, ux, uy ...
    float usq = ux*ux + uy*uy;
    float inv_tau = 1.0f / tau;
    #pragma unroll
    for (int k = 0; k < 9; ++k) {
        float cu = CX[k]*ux + CY[k]*uy;
        float feq = W[k] * rho *
            (1.0f + 3.0f*cu + 4.5f*cu*cu
             - 1.5f*usq);
        f_out[k*N+idx] =
            f[k] - inv_tau*(f[k]-feq);
    }
}

```

Figure 2: LBM iter-23 best, Opus (left) vs Gemini iter-13 best (right). Opus extracts A once, folds f_k^{eq} into two FMAs per direction and the relaxation into a third, and pins the threadgroup geometry; Gemini stays with the canonical BGK formula and the default geometry. In-distribution gmean: Opus 0.576 vs Gemini 0.553.

```

// Opus: Stockham radix-4, all in tg memory
for (uint s = 0u; s < stages4; ++s) {
    // load 4 inputs from cur[]
    float2 x0 = cur[b + 0u*Nq];
    float2 x1 = cur[b + 1u*Nq];
    float2 x2 = cur[b + 2u*Nq];
    float2 x3 = cur[b + 3u*Nq];
    // twiddle multiplies (skip s=0)
    if (s != 0u) { ... }
    // 4-point DFT, write to nxt[]
    threadgroup_barrier(mem_threadgroup);
    swap(cur, nxt);
}

```

```

// Gemini: simd_shuffle_xor for stages 1..5
// (no shared mem, no barrier)
if (logN >= 1u) {
    float2 v = simd_shuffle_xor(u, 1u);
    u = ((i & 1u) == 0u) ? (u+v) : (v-u);
}
// stages 2..4 analogous (xor 2, 4, 8)
if (logN >= 5u) {
    float2 v = simd_shuffle_xor(u, 16u);
    // twiddle, butterfly ...
    u = ((i & 16u) == 0u) ? (u+t) : (v-t);
}
// stages s>=6: fall back to tg memory
for (uint s = 6u; s <= logN; ++s) { ... }

```

Figure 3: FFT3D iter-10 best, Opus (left) vs Gemini iter-10 best (right). Opus implements Stockham radix-4 with threadgroup-memory ping-pong and a barrier per stage; Gemini exploits the 32-wide simdgroup to do the first five stages with `simd_shuffle_xor`, eliminating five barriers per 1D FFT. In-distribution gmean: Opus 0.167 vs Gemini 0.282.

regime where the pinned 32×2 geometry and the FMA-folded BGK extract every issued instruction — and that is the size with the largest absolute fraction-of-ceiling, so it dominates the gmean.

C.2 FFT3D: Gemini swaps the algorithm to `simd_shuffle_xor`

The `fft3d` gmean gap is larger (0.282 vs 0.167, $1.7\times$), and the diff is not a tighter version of the same kernel — Fig. 3 shows two different algorithms. Opus implements a textbook Stockham auto-sort radix-4 FFT: every stage ping-pongs through threadgroup memory, with a barrier between stages. Gemini observes that for the first five Cooley–Tukey stages the butterfly partner of lane i is at lane $i \oplus 2^{s-1}$ where $s \in \{1, \dots, 5\}$, and $2^{s-1} < 32$ — so it can be fetched with `simd_shuffle_xor`, no shared memory and no barrier. Only stages $s \geq 6$ fall back to threadgroup memory. The Apple GPU’s `simdgroup` width is exactly 32: this trick is Metal-specific (`simd_shuffle_xor` maps to a single hardware permute), and worth ~ 5 barriers per length- N FFT, ~ 15 per 3D transform. The same “find a different memory primitive” move shows up smaller-scale on `gradshaf`: both models converge on a `simdgroup`-tree max-reduction, but Gemini additionally casts `psi` to `float4*` and reads four scalars per transaction in the inner sweep, halving the number of issued loads on the dominant kernel.

D Related work

D.1 LLM-driven kernel-generation benchmarks

A wave of benchmarks has emerged for evaluating LLMs as generators of high-performance GPU kernels. *KernelBench* [11] targets PyTorch ML kernels (GEMM, attention, convolution, normalisation, activation) on CUDA and scores single-shot generation by speedup against the PyTorch eager baseline. *TritonBench* [7] extends to the Triton DSL and adds reference code-similarity to correctness and performance. *BackendBench* [13] evaluates correctness alone for kernels written against the PyTorch backend interface. *MultiKernelBench* [15] broadens the platform set to CUDA, Ascend C, and Pallas while retaining the ML-operator workload set. *NPUEval* [5] targets the AMD AIE NPU in C++ and exposes a tool-use feedback loop with multi-turn regeneration. Most recently, *KernelCraft* [8] benchmarks bare-metal assembly kernel generation on emerging accelerator ISAs (PLENA, AMD NPU, Coral NPU, Sonic BOOM), with native tool interfaces (`check_syntax`, `run_evaluation`, `view_output`, `grep_docs`) and a three-tier ML-task taxonomy (primitive, composite, end-to-end). Across these benchmarks the workload is ML and the headline number is speedup against a vendor or compiler baseline.

D.2 LLM-driven evolutionary code search

The broader paradigm of LLMs as *optimisers* inside evolutionary code-search loops was established by *FunSearch* [12] for symbolic-algorithm discovery and scaled by *AlphaEvolve* [9] across mathematical and algorithmic domains. *AI CUDA Engineer* [6] and *EvoEngineer* [4] specialise this paradigm to CUDA kernel optimisation, replacing single-shot or agentic regeneration with a population/archive driven by a fitness signal — closer in spirit to our ($\mu=1+\lambda=1$) loop, but on CUDA ML kernels rather than Apple Silicon scientific kernels.

D.3 Where METAL-SCI differs

Existing kernel-generation benchmarks share three structural choices that METAL-SCI deviates from. (i) *Workload domain*. All listed benchmarks target ML operators, where the optimisation surface is dominated by tiling and warp-level reduction patterns heavily represented in pretraining data. METAL-SCI targets scientific compute — regular and irregular stencils, all-pairs interactions, multi-field Boltzmann updates, neighbour-list molecular dynamics, parallel-chain MCMC, multi-kernel nonlinear PDE solvers, and butterfly spectral transforms — six structurally distinct optimisation regimes (R1–R6 of Section 2) whose canonical recipes [2, 10, 14, 1, 3] do not transfer between regimes. (ii) *Score basis*. All listed benchmarks normalise *achieved* against a vendor or compiler baseline (PyTorch eager, compiler-emitted code, expert hand-tuned reference). This couples the headline number to the strength of that baseline. METAL-SCI normalises against the per-chip roofline ceiling (peak FP32 GFLOPS, peak DRAM GB/s), a hardware-anchored target independent of any reference implementation, with a hard correctness gate ($S=0$ on tolerance failure) and a geometric mean across three problem sizes that discourages overfit to a single working-set regime. (iii) *Hardware*. No listed benchmark targets Apple Silicon’s Metal compute pipeline. Metal is structurally underrepresented in CUDA-centric pretraining data and surfaces a real out-of-distribution generalisation test on Metal-grammar and Metal-stdlib idioms — the `[[max_total_threads_per_threadgroup]]` attribute placement (Sec. 4, seven independent reproductions across five tasks within the opus-4-7 sweep, recurring in earlier opus-4-6 and gemini-3.1-pro runs), `half` as a reserved fp16 keyword, the absence of C++ lambdas, the availability of `simd_shuffle_xor` for intra-simdgroup butterflies, and the absence of `sincospi`. Table 3 summarises these axes against the comparator set.

E Further results: convergence and iteration budgets

The per-task iteration budget is uneven — lbm 25, wave3d 15, all others 10. Figure 4 shows the running self-speedup *best/seed* per iteration and justifies the choice. Two patterns recur. First, on most tasks (saxpy, heat2d, nbody, gradshaf, fft3d, hmc, ising, lj) the incumbent stops moving by iteration ~ 8 ; extending past 10 would not change the headline number. Second, lbm and wave3d are different: lbm’s Opus run plateaus at $1.36\times$ from iter 3 through iter 22, then breaks through

Table 3: METAL-SCI versus existing LLM kernel-generation benchmarks across the axes that drive the design of our harness. *Domain*: ML = neural-network operators (GEMM, attention, convolution, normalisation, activation); HPC = scientific compute (stencil, N -body, LBM, MD, MCMC, PDE solver, FFT). *Score basis*: what *achieved* is normalised against. *Multi-size*: whether the score requires generalisation across problem sizes. *Search*: the loop structure exposed to the LLM. Within the ML row, all listed benchmarks span a single optimisation regime (matmul-style, with norm/activation epilogues); METAL-SCI spans six structurally distinct regimes (R1–R6 of Section 2).

Benchmark	Target	Domain	Score basis	Multi-size	Search
KernelBench [11]	CUDA	ML	PyTorch speedup	—	single-shot
TritonBench [7]	Triton DSL	ML	speedup + code-sim	—	single-shot
BackendBench [13]	PyTorch backend	ML	correctness only	—	single-shot
MultiKernelBench [15]	CUDA / Ascend C / Pallas	ML	compiler speedup	—	multi-turn
NPUEval [5]	AIE C++ (AMD NPU)	ML	compiler speedup	—	tool-use
KernelCraft [8]	accelerator asm	ML	compiler speedup	cfg. vars	agentic tool-use
METAL-SCI (ours)	Apple Metal MSL	HPC	% of roofline	3 in + 1 held-out	evolutionary (1+1)

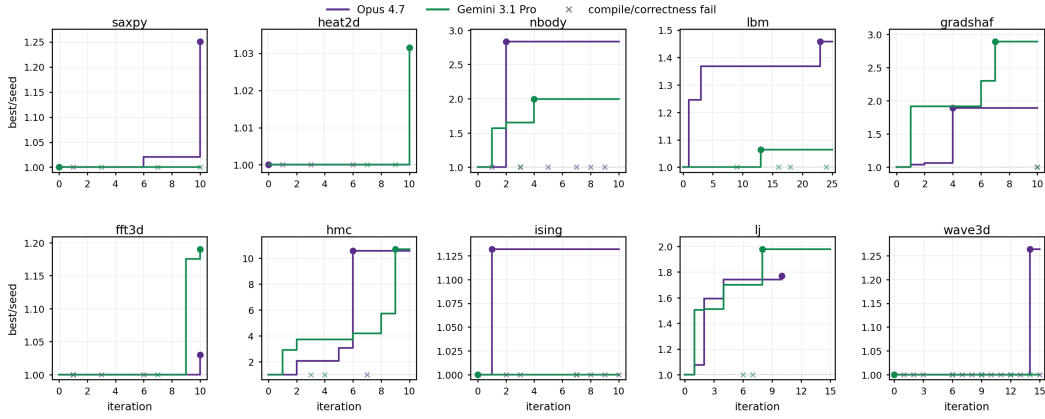


Figure 4: Per-task running self-speedup (best-so-far / seed) versus iteration, Opus 4.7 (purple) vs Gemini 3.1 Pro (green). Filled circles mark the iteration that achieved the final best; x along $y=1$ marks compile or correctness failures (which leave the incumbent unchanged). *lbm* and *wave3d* are the two tasks where extending the budget past 10 paid off: *lbm*’s Opus best lands at iter 23 ($1.46\times$, vs $1.36\times$ at iter 10), *wave3d*’s at iter 14 ($1.26\times$, vs $1.00\times$ at iter 10). Every other task plateaus by iter ~ 8 .

to $1.46\times$ at iter 23 (the BGK fold + pinned threadgroup of App. C); a 10-iter budget would have reported $1.36\times$ and missed App. C’s exemplar. *Wave3d*’s Opus run shows the same shape with the lift at iter 14. Both late-arriving wins land in tasks where the seed is already saturated on at least one size — the search has to pick a structural lever (kernel attribute, marching axis), and $1 + 1$ exploration finds those rarely. The figure also makes the silent-correctness story visible: every x on the $y=1$ baseline is a candidate that compiled or ran wrong, and Opus emits more of them than Gemini at every task with non-trivial search (*nbody*, *hmc*, *ising*, *lj*, *wave3d*).